DOCUMENT 3 / 3 — DONNÉES

Schéma de base de données

Modèle de données PostgreSQL : référentiel de cartes, prix & marché, utilisateurs, boutique physique — avec l'état réel de l'implémentation et les écarts connus par rapport au modèle cible.

PostgreSQL 16 | EF Core / Npgsql | Audit d'écarts inclus

Client
Gérant boutique Pokémon (30 m²)

Périmètre
Entités, relations, écarts schéma ↔ code

Source
Coffre Obsidian — 02-Architecture, 03-Metier

Généré le
31 août 2026

SOMMAIRE

Contenu du document

01 **Vue d'ensemble du modèle** — Carte complète des entités et de leurs relations

02 **Référentiel cartes** — Série → Set → Carte → Image

03 **Prix & marché** — Prix RAW, gradés, historique, ventes, population

04 **Auth & collection** — User, RefreshToken, UserCollection

05 **Boutique physique** — Shop, ShopProduct, ShopStock, ShopAppointment

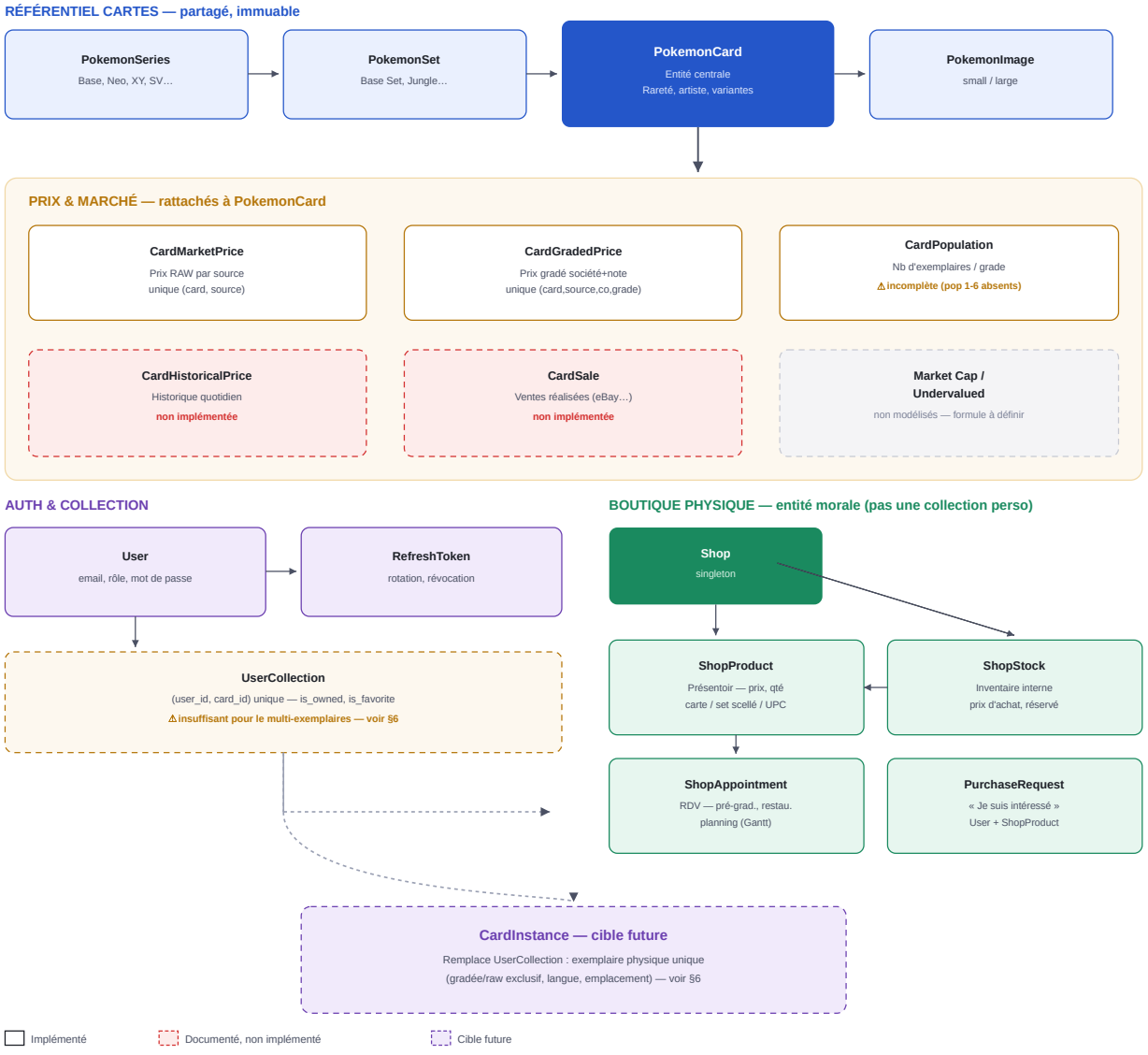
06 **Cible future — CardInstance** — Modélisation des exemplaires physiques (gradée/raw, langue)

07 **État des lieux — écarts connus** — Audit schéma vs implémentation réelle (2026-07-01)

01 — VUE D'ENSEMBLE

Carte du modèle de données

Quatre grandes zones : le référentiel de cartes (partagé, immuable), les données de marché associées, l'authentification et la collection utilisateur, et le back-office de la boutique physique. Une cible future (`CardInstance`) est identifiée mais pas encore implémentée.



Carte complète du modèle — voir le détail des colonnes par zone dans les sections suivantes.

02 — RÉFÉRENTIEL

Référentiel cartes

Hiérarchie **Série → Set → Carte → Image**, partagée et immuable : une carte n'existe qu'une seule fois en base, quel que soit le nombre d'utilisateurs qui la possèdent.

Table	Colonnes clés	Notes
PokemonSeries	`Id`, `PtcgSeriesId`, `Name`	Niveau le plus haut — ex. Base, Neo, EX, Sword & Shield, Scarlet & Violet
PokemonSet	`Id`, `SeriesId` → Series, `PtcgSetId`, `Name`, `Code`, `ReleaseDate`, `TotalCards`	Une extension — ex. Base Set, Jungle, Fossil
PokemonCard	`Id`, `SetId` → Set, `PtcgCardId`, `Name`, `CardNumber`, `Rarity`, `Artist`, `Types[]`, `Supertype`, `Subtypes[]`, `IsFirstEdition`, `IsShadowless`	Entité centrale du domaine — index sur `SetId`, `Name`, `Rarity`
PokemonImage	`Id`, `CardId` → Card, `SmallUrl`, `LargeUrl`	Une image par carte (small + large)

VOLUMÉTRIE CIBLE

Import PokémonTCG API visé à ≥ 90 % des cartes existantes (cible 100 %) — environ **15 000+ cartes**. C'est la brique la plus structurante encore à faire (voir doc « Vue d'ensemble » §5).

03 — MARCHÉ

Prix & marché

Deux tables de prix « courants » alimentées par le pipeline de scraping (voir doc « Architecture technique » §4), plus trois tables orientées historique / analyse — dont deux ne sont aujourd'hui que documentées.

Implémentées

Table	Colonnes clés	Unicité
card_market_prices	`card_id`, `source`, `variant`, `market_price`, `low_price`, `high_price`, `currency`, `fetched_at`	`(card_id, source)`
card_graded_prices	`card_id`, `source`, `grading_company`, `grade_value`, `price`, `currency`	`(card_id, source, grading_company, grade_value)`
CardPopulation	`CardId`, `GradeCompany`, `Grade`, `Population`, `LastUpdated`	`(CardId, GradeCompany, Grade)`

CHAQUE SOCIÉTÉ DE GRADATION RESTE INDÉPENDANTE

Pas de mapping BGS/CGC/SGC → PSA : une carte BGS 9.5 a sa propre ligne, distincte d'une PSA 9. Le champ `grading_company` est un `string` libre côté `CardGradedPrice` (flexible, pas de migration à chaque nouvelle société) mais un `enum` côté `CardPopulation` — incohérence mineure entre les deux tables, à harmoniser si une société supplémentaire est ajoutée.

Documentées mais non implémentées

Table	Rôle prévu	Bloque
CardHistoricalPrice	Historique quotidien (moyennes 7j/30j/90j, volumes, min/max) — partitionnable par année	Graphique d'historique de prix (façon TradingView / CardLadder)
CardSale	Ventes réalisées individuelles (eBay, CardMarket) — prix, date, source, titre	Last Sold Price/Date, Rate of Growth, variations 24h/7j/30j

POURQUOI C'EST IMPORTANT

La donnée source existe déjà côté fournisseurs (CardLadder expose `dailySales` et des séries temporelles via Firestore) — il manque le **pipeline d'ingestion et les tables pour la persister**, pas la source elle-même. Sans ces deux tables, le panneau détail carte ne peut afficher ni Market Cap, ni indicateur « Undervalued », ni graphique d'historique fiable — voir §7 pour le détail de l'impact sur l'écran Bourse.

04 — UTILISATEURS

Auth & collection

Table	Colonnes clés	Notes
User	`Id`, `Email` (unique), `PasswordHash`, `Username`, `Role`, `IsActive`	Rôles : `admin` / `employee` / `client`
RefreshToken	`Id`, `UserId` → User, `Token` (unique), `ExpiresAt`, `IsRevoked`, `ReplacedBy`	Rotation à chaque `/auth/refresh` — voir doc Architecture §3
user_collection	`user_id` → User, `card_id` → PokemonCard, `is_owned`, `is_favorite`, `is_priority`	Unique `(user_id, card_id)` — collection personnelle

ÉCART CONNU — UN SEUL EXEMPLAIRE REPRÉSENTABLE PAR CARTE

La contrainte d'unicité `(user_id, card_id)` ne permet pas de représenter plusieurs exemplaires physiques d'une même carte (une RAW + une gradée, ou deux langues différentes). C'est la limite qui motive la table cible `CardInstance` détaillée en §6. Vente : `user_collection` porte aussi `for_sale` et `sale_price` côté admin (catalogue boutique) — un index partiel `WHERE for_sale = TRUE` optimise le catalogue client.

05 — BOUTIQUE

Boutique physique

Modèle propre à l'instance Pessac : le back-office de la boutique en tant qu'entité morale, distinct de la collection personnelle d'un utilisateur.

Table	Colonnes clés	Notes
Shop	Nom, description, adresse, lien Google Maps, horaires, services, photos, réseaux sociaux	Singleton — une seule boutique par déploiement
ShopProduct	`type` (card_raw / card_graded / sealed_set / sealed_upc), `card_id` ?, `condition`, `grading_company` ?, `grade_value` ?, `price`, `quantity`, `is_available`	Présentoir — quantité à 0 ⇒ retrait automatique du catalogue client
ShopStock	`type`, `card_id` ?, `quantity_total`, `quantity_available`, `quantity_reserved`, `purchase_price`	Inventaire interne — distinct du présentoir, sert au calcul de marge
ShopAppointment	`client_name`, `client_email`, `service_type` (pre_grading / restoration / other), `scheduled_at`, `status`, `assigned_to` ?	RDV — voir écart d'appartenance ci-dessous
PurchaseRequest	`shop_product_id` → ShopProduct, `user_id` → User, `message` ?, `status` (pending/contacted/completed/cancelled)	Créée par le bouton « Je suis intéressé »

ÉCART SPEC ↔ IMPLÉMENTATION — APPARTENANCE DES RDV

`ShopAppointment` n'a **pas** de colonne `UserId` : l'appartenance à un client se fait via `ClientEmail`, copié depuis le compte à la création. À faire évoluer vers un `UserId` si le module Boutique en ajoute un. Autre point : aucune notification email n'est envoyée en V1 (RDV et demande d'achat sont seulement loggués côté serveur) — prévu en V2.

06 — CIBLE FUTURE

CardInstance — exemplaire physique

Règles métier définies (2026-07-10), implémentation à venir. Distingue le **référentiel** (la carte en tant qu'objet défini, partagé) de l'**instance** (un exemplaire physique précis, propre à un utilisateur ou à la boutique).

Concept	Ce que c'est	Exemple
PokemonCard	Le référentiel — objet de collection défini, unique en base	Charizard Base Set #4/102
CardInstance	Un exemplaire physique, propre à son détenteur — N possibles par carte	Mon Charizard PSA 9 EN, acheté 420 €

Règle 1 — Gradée OU Raw, jamais les deux

Type	GradingCompany	GradeValue	Condition	Language	Valide ?
Raw	—	—	NM	EN	✓
Raw	PSA	10	NM	EN	✗ les deux remplis
Graded	PSA	9	—	JP	✓
Graded	—	—	—	FR	✗ société/note manquantes
Raw	—	—	—	EN	✗ condition manquante

États Raw affichés en français : NM → Neuf, LP → Très bon, MP → Bon, HP → Usé, DMG → Abîmé (code technique inchangé en base, seul l'affichage est traduit).

Règle 2 — Langue de la carte ≠ langue de l'interface

L'interface reste 100 % française quelle que soit la langue de la carte consultée. La langue est un attribut physique immuable de l'instance (EN, FR, JP, DE, ES, IT, KO) et **détermine la valeur de marché** — même carte, même grade, prix différent selon la langue. Un sélecteur de langue est obligatoire à la création d'une instance.

Règle 3 — Multi-instances autorisées

EXEMPLE — UN MÊME RÉFÉRENTIEL, CINQ EXEMPLAIRES

Charizard Base Set #4/102 → Raw NM EN (~95 €) · Raw LP FR (~220 €, prime FR) · PSA 9 EN (~450 €) · PSA 10 EN (~2 400 €) · PCA 9 FR (~380 €). Chaque instance a son propre emplacement, prix de revient, prix de vente et statut (collection / à vendre / en boutique / en RDV).

Modèle cible

```
CardInstance
  Id, UserId, CardId (→ PokemonCard)
  Type            : Graded | Raw            -- discriminateur exclusif
  GradingCompany? : string                  -- si Graded (PSA/BGS/CGC/PCA/SGC)
  GradeValue?     : decimal                 -- si Graded (1-10)
  Condition?      : string                  -- si Raw (NM/LP/MP/HP/DMG)
  Language        : string                  -- EN|FR|JP|DE|ES|IT|KO, toujours renseignée
  LocationId?                               -- emplacement physique (classeur, présentoir...)
  PurchasePrice?, SalePrice?
  Status          : Collection | ForSale | InDisplay | InAppointment
```

IMPACT SUR LA RECONNAISSANCE VISUELLE (SCAN IA)

Une carte gradée doit être représentée **avec sa slab** (PSA rouge/blanc, BGS noir/or, CGC bleu, PCA blanc/marine, SGC noir/blanc) dans la collection et la PLV boutique — l'acheteur identifie le statut avant même de lire la note. Le scan IA doit détecter la présence d'une slab, identifier la société par sa couleur, lire la note, et pré-remplir automatiquement le formulaire en conséquence.

07 — AUDIT

État des lieux — écarts connus

Audit du 2026-07-01, réalisé suite à la refonte du panneau détail carte (écran Bourse, inspiration CardLadder) — vérifié directement dans le code (`Pokekool.Domain`, `Pokekool.Infrastructure/Migrations`), pas seulement dans la documentation.

Impact concret sur le panneau carte actuel

Champ affiché	Source actuelle	État
Prix marché par grade	`card_graded_prices.price`	Réel
Date de mise à jour	`MAX(fetched_at)`	Réel
Prix bas par grade	codé en dur `null`	Jamais rempli — colonne `low_price` absente
Population par grade	codé en dur `null`	Bug de jointure — `CardPopulation` jamais jointe
Last Sold, Market Cap, Rate of Growth, historique, annonces actives	—	Non implémentés — pas de données réelles en base

Détail des écarts

#	Sujet	Constat
1	card_graded_prices	Colonnes `low_price`, population, `variation` absentes de la table réelle malgré la doc
2	CardPopulation	Stocke seulement `TotalGraded, Pop10..Pop7, GemRate10` — pas de Pop1 à Pop6 ; jointure jamais faite dans le handler
3	Historique & ventes	`CardHistoricalPrice` et `CardSale` : aucune entité, migration ni repository — décrites en doc, absentes du code
4	Market Cap	Aucun champ, aucune formule définie — proposition : population(grade) × prix(grade) sommé, calculé ou matérialisé
5	Indicateur « Undervalued »	Aucun champ, aucune règle — dépend d'abord de l'historique (#3)
6	Annonces actives	Pas de cache pour les listings eBay/CardMarket « Buy it now » — à trancher (live vs table `card_active_listings`)
7	Dimension langue	`card_market_prices/graded_prices/populations` sans colonne `language` — tout est implicitement EN
8	user_collection	Pas de grade/quantité/langue — un seul exemplaire représentable par `(user_id, card_id)`, voir §6
9	Stockage physique	Tables `storage_rooms/boxes/binders/locations/movement_events` proposées mais non créées

Notes de performance (prévues)

- Partitionnement par année pour `CardHistoricalPrice` (volumétrie potentiellement en millions de lignes)
- Recherche fulltext `pg_trgm` sur `PokemonCard.Name` pour la recherche instantanée (< 200 ms visé)
- Redis envisagé pour le cache des prix actuels (lecture intensive)
- VACUUM régulier sur `CardSale` une fois implémentée (writes fréquents)

Sources — coffre Obsidian « Projet_Pokemon » : 02-Architecture/Schema-BDD.md, 03-Metier/Entites-Domaine.md, 03-Metier/Regles-Instance-Physique.md, 03-Metier/Module-Boutique-Physique.md. Audit d'écarts daté du 2026-07-01. Document généré le 31 août 2026.